

Clase 04

Grid - Diseño responsive

Grid

El **CSS grid** se puede utilizar para lograr muchos diseños diferentes.

También se destaca por permitir **dividir una página** en áreas o regiones principales, por definir la relación en términos de **tamaño, posición y capas** entre partes de un control construido a partir de primitivas HTML.

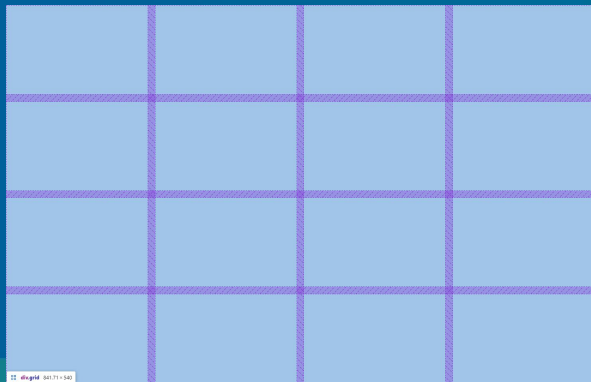
```
<section class="grid">
  <div>...</div>
  <div>...</div>
  <article>...</article>
</section>

.grid {
  display: grid;
}
```

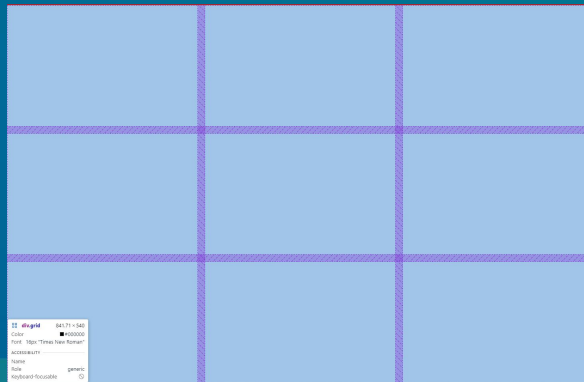
Grid

Se pueden lograr grillas de distintas formas y tamaños. Algunos ejemplos:

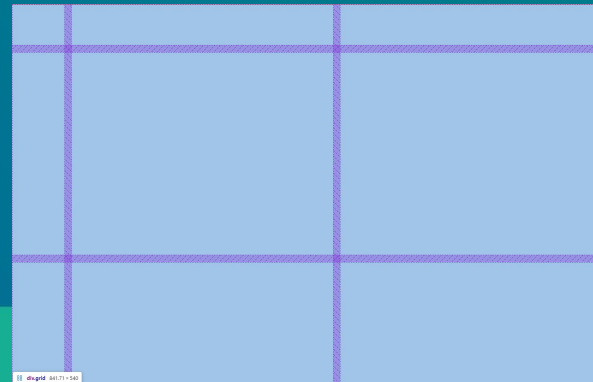
4 filas, 4 columnas



3 filas, 3 columnas



3 filas, 3 columnas, asimétrico



Grid

Se pueden lograr grillas de distintas formas y tamaños.

Para ello, necesitamos tener bien definidos dos aspectos:

1. Nuestro objeto contenedor, quién va a definir los parámetros de la grilla.
2. Los objetos hijos, que van a ser colocados dentro de la grilla.

Grid

`<!-- Pensemos una grilla de
4 elementos.`

`Dos filas, dos columnas -->`

```
<div class="grid">  
  <div class="uno"></div>  
  <div class="dos"></div>  
  <div class="tres"></div>  
  <div class="cuatro"></div>  
</div>
```

grid-template-columns
grid-template-rows

Permiten determinar la forma de la grilla, especificando cómo serán sus filas y columnas.

Una forma de hacerlo, es especificar el tamaño de cada fila/columna en orden.

Grid

`<!-- Pensemos una grilla de
4 elementos.`

`Dos filas, dos columnas -->`

```
<div class="grid">  
  <div class="uno"></div>  
  <div class="dos"></div>  
  <div class="tres"></div>  
  <div class="cuatro"></div>  
</div>
```

grid-template-columns
grid-template-rows

El siguiente código CSS dice:
La primera columna medirá el 50% del espacio.
La segunda columna medirá el 50% del espacio.

```
.grid {  
  display: grid;  
  grid-template-columns: 50% 50%  
}
```

Grid

`<!-- Pensemos una grilla de
4 elementos.`

`Dos filas, dos columnas -->`

```
<div class="grid">  
  <div class="uno"></div>  
  <div class="dos"></div>  
  <div class="tres"></div>  
  <div class="cuatro"></div>  
</div>
```

grid-template-columns
grid-template-rows

Finalizamos repitiendo lo mismo para las filas.
El resultado debería dividir la grilla en 4 partes iguales.

```
.grid {  
  display: grid;  
  grid-template-columns: 50% 50%  
  grid-template-rows: 50% 50%  
}
```

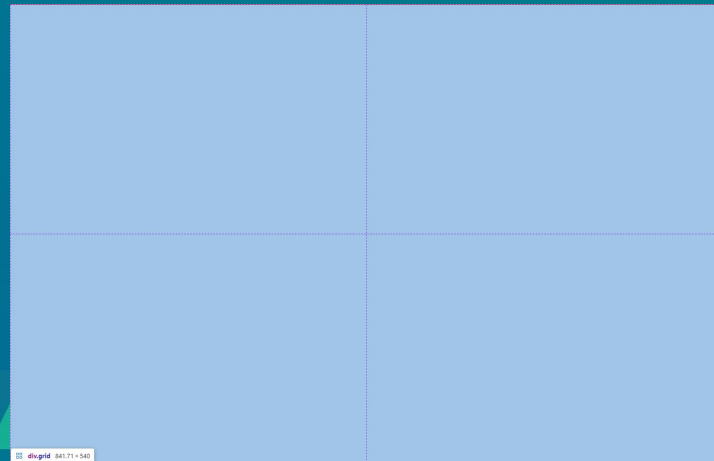
Grid

grid-template-columns
grid-template-rows

Con esto se logra dividir al contenedor en 4 partes iguales, como fue pensado.

Existen muchas alternativas.

```
.grid {  
  display: grid;  
  grid-template-columns: 50% 50%  
  grid-template-rows: 50% 50%  
}
```



Grid

Estos son algunos ejemplos que pueden probar. No son todos.

Una vez que logramos lograr la grilla, es momento de ubicar a los elementos hijos.

```
.grid {  
  display: grid;  
  grid-template-columns: 25% 100px;  
  grid-template-rows: 100px 100px;  
}
```

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  grid-template-rows: repeat(3, 1fr);  
}
```

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 1fr 1fr;  
}
```

Grid

grid-row - grid-column

Son las propiedades utilizadas para definir en qué fila y columna va a estar ubicado un elemento. En primera instancia, podemos indicar el número de fila o columna para ubicarlos.

```
<div class="grid">
  <div class="uno"></div>
  <div class="dos"></div>
  <div class="tres"></div>
  <div class="cuatro"></div>
</div>
```

```
.uno {
  grid-row: 1;
  grid-column: 1;
}
```

```
.dos {
  grid-row: 1;
  grid-column: 2;
}
```

```
.tres {
  grid-row: 2;
  grid-column: 1;
}
```

```
.cuatro {
  grid-row: 2;
  grid-column: 2;
}
```

Grid

grid-row - grid-column

Alternativas: Una misma celda puede ocupar varios espacios.

Atención a la class "tres".

```
<div class="grid">  
  <div class="uno"></div>  
  <div class="dos"></div>  
  <div class="tres"></div>  
</div>
```

```
.uno {  
  grid-row: 1;  
  grid-column: 1;  
}
```

```
.dos {  
  grid-row: 1;  
  grid-column: 2;  
}
```

```
.tres {  
  grid-row: 2;  
  grid-column: 1 / 2;  
}
```

Grid

Otras propiedades para el elemento padre:

`grid-gap`

Permite añadir espacio entre cada celda.

`grid-auto-rows - grid-auto-columns`

Hará que las filas y/o columnas se ajusten automáticamente al contenido. Es más útil cuando no conocemos cuál será el tamaño final de la grilla.

Grid

Documentación de utilidad

https://developer.mozilla.org/es/docs/Web/CSS/CSS_grid_layout

Grid

¿Dudas o consultas?